ROBOTICS SOFTWARE

박희연

ROS 2 기반 주행과 위치 보정을 다루는 로봇 소프트웨어 개발자

삼육대학교 컴퓨터공학과 · 2026년 8월 졸업

heeyeon5877@gmail.com

github.com/heeyeon5877-tech

01 · 물류센터 다중 AMR ● 진행 중

7인 팀 프로젝트 · 1 / 2 · 주행 환경과 정렬 설계

주행은 Nav2로, 정밀 정렬은 LiDAR·ArUco로

초기 다중 로봇 Nav2 구성(팀 공동)과 LiDAR·ArUco 기반 위치 보정·정렬 담당 · 핵심 정렬 기능 구현 완료, 실물 통합 검증과 최종 튜닝 진행 중

기여 · 주행 환경 구성(팀 공동) → 위치 보정·정렬(담당)

다중 로봇 Nav2 주행 환경 구성 (팀 공동)

팀원들과 함께 TurtleBot3 3대의 namespace·TF 접두사·Nav2 파라미터와 센서·명령 브리지를 분리했고, AMCL 초기 위치와 cmd_vel 전달 흐름을 정리해 RViz 목표 주행을 확인했습니다.

Nav2 주행 문제 추적

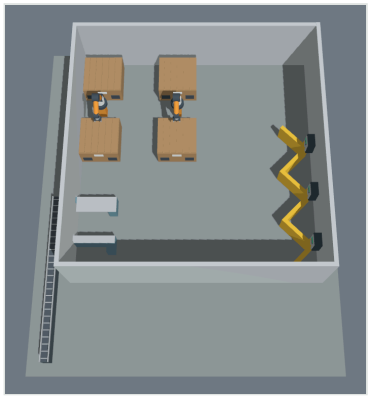
lifecycle inactive 상태와 지도상 장애물에 걸린 시작 위치를 확인해 목표 주행이 멈추는 원인을 분리했습니다.

Action 기반 LiDAR·ArUco 정렬·보정

Action 요청 시 카메라를 실행하고, LiDAR 벽 직선과 ArUco 마커 방향각으로 방향·좌우·거리를 보정합니다. 정렬 완료 측정값으로 map→base_footprint 위치를 계산해 /initialpose로 발행하고, AMCL/Nav2 위치를 보정한 뒤 카메라를 종료합니다.

CPU 부하 대응

Action 요청 시에만 카메라를 실행·종료하고, 320×240 캘리브레이션 적용과 TF listener 실행 분리로 부하를 줄였습니다.



Gazebo 다중 로봇 환경 · 시뮬레이션 검증 (실물 3대 동시 주행은 미검증)

팀원 담당 · Route Graph 완성·Route Server, 후반 노드 주행, 도킹, 미들웨어·웹 관제·배차 (개인 참여: Route Graph GeoJSON 노드 좌표 초안까지)

01 Nav2 주행	02 정렬 Action	03 위치 보정
작업 노드 근처까지 이동	카메라 시작 · LiDAR 벽 직선 + 마커 방향각 보정 반복	/initialpose 발행 · 카메라 종료

스킬 & 사용 프로그램

프로젝트 코드 · github.com/E1I6-Logistics/Logistics_AMR

언어 · 개발 기술

Python · ROS 2(Jazzy)
Nav2 · AMCL · TF2 · pytest

사용 프로그램

Gazebo · RViz
OpenCV(ArUco) · GeoJSON

로봇 · 구현 환경

TurtleBot3 3대 시뮬레이션
실물 TurtleBot3(LiDAR·카메라 정렬 테스트)

실물 테스트로 실패 원인을 분리하고 정렬 안정성을 개선

실물 TurtleBot3 3대 · 4개 노드 · Nav2·Zenoh·카메라 동시 실행(CPU load 3.8~27.8, 4코어) · 로봇 ROS 로그·오도메트리 기록 기준

153/169회	5/5회	0.3 / 0.3 cm · 0.5°
정렬 성공 · 1·2차 누적 (2차 테스트 71/77회)	실패 원인을 반영한 최종 코드 정렬 성공 · 46~84초	2차 테스트 성공 런 최대 잔차 · 좌우 / 거리 / yaw

관찰 → 원인 → 수정 → 재시험

마커 x/z → LiDAR 벽 직선 + 마커 방향각

마커 x/z가 0.2 cm 안으로 맞아도 실제로는 약 7°·2~3 cm 틀어진 것을 줄자로 확인했습니다(각 1회). LiDAR 벽 직선으로 yaw·거리를, 마커 방향각으로 좌우를 구하고, 목표 근처는 3회 중간값으로 판정합니다.

Nav2 부하 중 시간 초과

load 7~19에서 불필요한 후진·측정 흔들림을 로그로 찾아 후진 금지·중간값 판정으로 바꿔, robot1 4개 노드 17/17 성공, 비슷한 시작 위치의 N5 47~80초 → 34~65초로 줄였습니다.

직진 쏠림으로 좌우 보정이 넘어감

robot2가 10 cm 직진에 1.3°씩 돌아 좌우 보정이 반대쪽으로 넘어갔습니다. 벽과 평행한 절대 방향 기준으로 회전하게 바꿔, 일부러 밀어낸 6회가 모두 22~44초에 성공했습니다.

회전 정지 후 추가 회전

명령 지연(약 0.3초)과 관성으로 0.6~1.2°를 더 돌았습니다. 멈춘 뒤 남은 각도만 최소 속도로 시간 기반 회전하게 바꿔 잔여 각도를 0.35° 이하(odom 기준)로 줄였습니다.

카메라 프로세스 잔존

Action 종료 뒤 남은 카메라 프로세스를 다음 Action이 재사용해 시간 초과가 났습니다(4회). 프로세스 그룹 정리(SIGTERM → SIGKILL)를 넣은 뒤 5회 모두 남은 프로세스가 없었습니다.

노드별 정렬 결과 (성공/시도) · 1차 테스트 최대 잔차 0.5 / 0.5 cm · 1.0°

노드	1차	2차	합계	실패 원인
N5 ID24+29 · 벽, 정면 로봇팔	36/41	7/7	**43/48**	튜닝 중 시간 초과 3, 로봇팔이 카메라 가림 2
N6 ID25+28 · 벽, 정면 로봇팔	10/10	21/24	**31/34**	카메라 프로세스 잔존 3
N3 ID27 · 컨베이어·로봇팔	14/16	27/28	**41/44**	목표값 입력 오류 1, 부하 중 시간 초과 1, 전압 부족으로 LiDAR 스캔 부족 1
N4 ID26 단일	5/6	—	**5/6**	로봇팔 설치 후 ID26 가림 → ID23 추가
N4 ID23+ID26	17/19	16/18	**33/37**	1차: LiDAR 미실행, 배터리 리셋 · 2차: 직진 쏠림 1, 카메라 잔존 1

검증 범위

잔차는 로봇 자체 측정값이며 튜닝 런과 일부러 밀어낸 런이 포함됩니다. 줄자 확인은 최종 설정에서 했고, robot3 N5는 2차 테스트에서 다시 돌리지 않았습니다. 카메라 정리 분기가 실제로 실행되는 장면과 수정 전·후 같은 시작 위치 비교는 아직 확인하지 못했습니다.

남은 문제

2차 테스트 실패 6건은 전압 부족·직진 쏠림·카메라 잔존으로 분리해 제어·프로세스 문제를 수정했습니다. robot3 전원 점검과, 이동 후 odom과 LiDAR 기준 방향이 1.6~4.2° 차이 나는 현상은 남아 있습니다.

EKF 적용 전 오도메트리 기준 측정 · 본인 수행

0.5 m 왕복 복귀 차이 12.51 ± 5.43 mm(n=6), 360° 회전 6회 모두 부족. 팀원의 EKF 비교: odom 위치 오차 12.19 → 6.03 cm, 실제 복귀 11.50 → 11.10 cm.

다음 단계 · robot3 전원 점검 · odom–LiDAR 방향 차이 대응 · 수정 전·후 같은 시작 위치 반복 비교 · Zenoh 경유 Action 통합 · 정렬 제어 고도화

산업 현장 보호구 착용 감지

객체 탐지(OD)와 다중 라벨 분류(MLC)를 비교할 데이터를 구성하고, 비교 결과를 실제 판정 흐름에 반영했습니다.

담당 · 데이터 전처리 / 시스템 통합 / 대시보드

탐지용 데이터를 사람 단위 분류 데이터로 변환

약 22,000장의 원본에서 사람 검출·크롭과 PPE 라벨 매칭을 자동화하고, 겹침·저해상도·라벨 충돌 데이터를 제거했습니다.

클래스 불균형과 평가 데이터 누수 대응

소수 상태는 모두 유지하고 다수 상태는 최대 1,000장으로 샘플링했습니다. 독립 평가용 원본 100장의 파생 크롭은 학습에서 격리했습니다.

하이브리드 PPE 판정

1명은 MobileNetV3 분류(MLC), 2명 이상은 YOLO 객체 탐지(OD)로 분기합니다. 사람 수만큼 반복되는 분류 추론의 속도 저하를 고려했습니다.

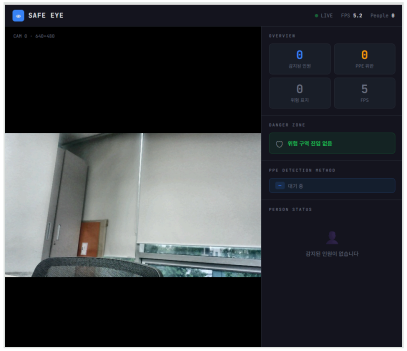
대시보드와 모듈 통합

config·camera·detector를 분리해 팀원 모델을 같은 입출력 흐름으로 연결하고, 감지 인원·위반·위험 구역·FPS를 한 화면에 표시했습니다.

22,000장
전처리 대상 원본(약)

70:15:15
학습·검증·테스트 분할

100장
독립 평가용 원본 격리



Safe Eye 통합 대시보드 · 감지 인원·위반·위험 구역·FPS 패널

독립 GT 평가셋 기준 PPE 판정 정확도 (팀 비교 실험, 사람 단위)

환경	헬멧 OD	헬멧 MLC	조끼 OD	조끼 MLC	FPS (PC 측정)	채택
1인 n=588	92.6%	91.6%	79.6%	**90.8%**	MLC 3.48 / OD 3.70	**MLC**
3인 이상 n=889	92.8%	89.6%	73.6%	77.8%	MLC 1.92 / OD 3.70	**OD**

판정 방식을 선택한 근거

1인 환경에서는 조끼 정확도가 MLC 90.8%로 OD보다 11.2%p 높고 속도 차이는 작았습니다. 3인 이상에서는 조끼 정확도 차이가 4.2%p로 줄어든 반면 MLC 속도가 절반 수준으로 떨어져, 실시간성을 위해 OD를 선택했습니다.

스킬 & 사용 프로그램

프로젝트 코드 · github.com/heeyeon5877-tech/safety_robot

언어 · 데이터 처리
Python · 데이터 전처리
다중 라벨 분류 · 평가셋 구성

모델 · 실행 기술
YOLO · MobileNetV3
TFLite

대상 장치 · 통합 화면
Raspberry Pi 5
Safe Eye 대시보드

센서와 상태 표시를 연결한 RC카

STM32F411RE 기반 무선 RC카 팀 프로젝트에서 IMU·초음파 센서 입력과 장애물 감지 로직을 담당하고, 센서 상태값이 LCD 표시 및 팀의 무선 조종·모터 제어 흐름에 연동되도록 통합했습니다.

담당 · IMU 입력 / 초음파 장애물 감지 / 센서 상태값 LCD 연동

초음파 장애물 감지

HC-SR04의 Echo 펄스를 타이머 Input Capture로 측정했습니다. 10 cm 이내 장애물을 감지하면 전진 목표 RPM을 0으로 제한하고 후진·회전은 허용했습니다.

IMU 기울기 입력

ADXL345의 가속도 값을 I2C로 읽어 차체 기울기를 감지했습니다.

IMU 상태값 LCD 연동

IMU 값을 50 ms 주기로 읽어 LCD 표시 흐름에 연동해, 차체 기울기가 화면에 바로 나타나도록 했습니다(LCD 기본 동작은 팀원 담당).

센서·모터 결합 후 오동작 해결

기존 코드와 비교하고 개별 동작을 확인한 뒤 납땜·연결 오류를 수정했습니다. 통신 응답뿐 아니라 실제 센서값·출력 동작과 물리적 배선을 함께 검토해야 한다는 점을 배웠습니다.

10 cm
전진 제한 기준

50 ms
IMU 읽기 주기

100 ms
초음파 트리거 주기

통합 구조 · 1 ms 타이머(TIM5) 기준 주기 분리

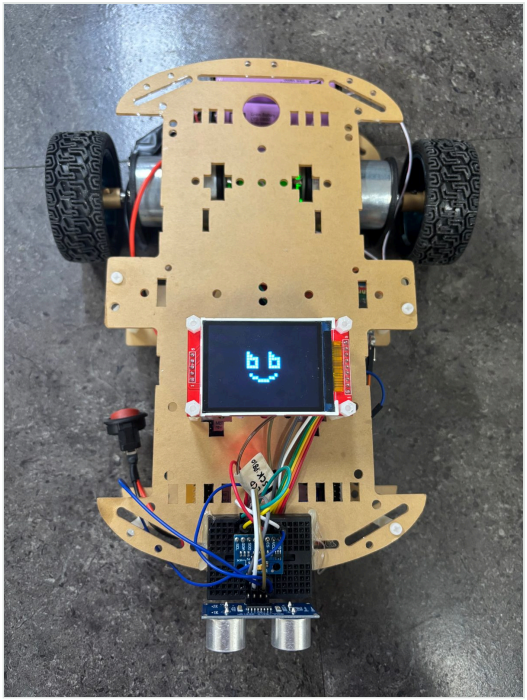
펌웨어 처리 주기 · **굵은 글씨**는 본인 담당 센서 처리

주기	처리 내용
10 ms	목표 RPM · **장애물 조건** · 엔코더 · 모터 제어
50 ms	패킷 확인 · **IMU 읽기** · LCD 표시 위치 갱신(IMU 값 연동)
100 ms	**초음파 트리거**

타이머 인터럽트에서 플래그를 설정하고 메인 루프에서 처리하는 구조로 센서 읽기와 표시·통신 주기를 분리했습니다.

팀 통합 결과

무선 조이스틱 입력부터 모터 제어까지 연결하고, 10 cm 장애물 정지·500 ms 통신 단절 정지·IMU 연동 LCD 표시를 실물 주행으로 시연했습니다.



실제 제작한 RC카 · IMU 연동 LCD 화면 · 팀 발표자료 사진

스킬 & 사용 프로그램

프로젝트 코드 · github.com/heeyeon5877-tech/Lcd_RC_Car_With_STM32

언어 · 제어 기술

C · HAL · I2C
Input Capture · 타이머 인터럽트

사용 프로그램

STM32CubeMX

보드 · 센서

STM32F411RE · ADXL345
HC-SR04 · LCD

자동화 설비 HMI와 서보 기본 제어

운용 화면을 설계하고, 조작 입력과 PLC 제어·상태 정보를 연결했습니다.

담당 · HMI 화면 설계 / 기본 기능 구현

조작과 상태 확인이 가능한 화면을 구성했습니다.

HMI 3개 화면 설계

메인·서보 컨트롤·환경 컨트롤로 구분하고, 한·영 전환과 위치·속도·에러 상태 표시를 구성했습니다.

서보 기본 동작 연결

원점 복귀, 위치 지정, JOG 이동·속도 변경, 에러 리셋, 동작 시작과 비상정지·해제 기능을 구현했습니다.

통합 오류 해결

HMI 태그 연결 오류를 수정하고, JOG 버튼 미입력 시에도 속도값이 인가되던 문제에 비동작 조건을 추가했습니다.

역할 분담 · 창고 층 선택, Tact Time 계산과 적재 램프 응용 로직은 팀원 담당



서보 컨트롤 · 조작 입력과 상태 표시



환경 컨트롤 · 설비 동작·적재 현황

스킬 & 사용 프로그램

프로젝트 코드 · github.com/heeyeon5877-tech/PLC_HMI_Servo_Control

제어 · 화면 설계

HMI 설계 · 서보 기본 제어
태그 연동 · Ethernet

사용 프로그램

GX Works2
GT Designer3

PLC · 서보 장치

Mitsubishi Q03UDV
MR-J4